

Domain 3: Claude Code Configuration & Workflows — Self-Questioning List

CCAR-F Prep — Domain 3 (20% of scored content).

A. The CLAUDE.md Configuration Hierarchy

1. What is CLAUDE.md meant to be — a README for humans, or something else entirely?
2. What three levels make up the CLAUDE.md hierarchy, and what does the @imports mechanism add on top of them?
3. Why are "which level to use" and "CLAUDE.md is advisory, not guaranteed" described as two of the most frequently tested concepts in Domain 3?
4. What happens at the start of every Claude Code session regarding CLAUDE.md files — are they loaded selectively or merged together?

Root CLAUDE.md

5. What kind of content belongs in the root CLAUDE.md — project-wide conventions, or personal preferences?
6. Why should the root CLAUDE.md include patterns that "might otherwise look like anti-patterns" (e.g., force-unwrapping in test files)?
7. What test should content pass before being added to root CLAUDE.md — does it materially change Claude's decisions, or can Claude infer it from reading the code?
8. Why does Anthropic's guidance suggest that having more than 15 rules likely means the genuinely load-bearing ones haven't been identified?

Subdirectory CLAUDE.md Files

9. What happens when Claude works on files inside a directory that has its own CLAUDE.md — does the subdirectory file replace the root file, or supplement it?
10. Why would placing separate CLAUDE.md files in `/terraform/` and `/kubernetes/` avoid "mixing unrelated conventions"?
11. What happens when the root and a subdirectory CLAUDE.md contain conflicting instructions?

User-Level CLAUDE.md

12. What kind of content belongs in `~/.claude/CLAUDE.md` rather than the project's root CLAUDE.md?
13. Why is the user-level file "never committed to version control," and how does it combine with project-level files at session start?
14. Given a scenario describing an instruction that should apply to the whole team and be version-controlled, which CLAUDE.md file is correct — and how does that change if the instruction is a personal preference instead?

@imports for Shared Standards

15. What problem does the `@imports` syntax solve in a monorepo with multiple packages needing overlapping but not identical standards?
16. In the auth-package vs. notifications-package example, why does each package only import the standards relevant to it rather than all shared standards?
17. What is the maintainability advantage of `@imports` over duplicating standards across every package's CLAUDE.md?
18. What is the key difference between `@imports` (which relies on each maintainer knowing what to import) and `.claude/rules/` (which uses path patterns to auto-load)?

Advisory Nature of CLAUDE.md

19. Why is CLAUDE.md described as advisory rather than deterministic — what does Anthropic's documentation say Claude treats its content as?
20. Does adding emphasis markers like "IMPORTANT" or "YOU MUST" guarantee 100% compliance, or does it only improve the probability?
21. If a rule must be followed every single time without exception, what kind of mechanism is actually required instead of CLAUDE.md?

The /memory Diagnostic Command

22. What does the `/memory` command reveal, and why is it "the most efficient first diagnostic step" when Claude inconsistently follows a convention?
23. In the ApiError-vs-generic-try/catch scenario, what does it mean if `/memory` shows the file isn't loaded at all — versus if it shows the file *is* loaded but still isn't consistently followed?
24. When a question describes inconsistent convention-following "across different coding sessions" and asks for the most efficient first diagnostic step, why is jumping straight to more examples or path-specific rules the wrong move?

B. Path-Scoped Rules with `.claude/rules/`

25. What advantage does `.claude/rules/` offer over placing the same instructions in a subdirectory CLAUDE.md?
26. What happens to a rule file's content when Claude is working on files that do *not* match its path pattern — does it stay loaded, or drop out of context entirely?
27. Are `.claude/rules/` files deterministic enforcement, or do they share the same advisory nature as CLAUDE.md?

YAML Frontmatter Path Patterns

28. In the Terraform rule example (`paths: ["terraform/**/*"]`), what happens when Claude edits a Kubernetes manifest instead?
29. Why would a project split rules into separate files for Terraform, Kubernetes, TypeScript, and CI/CD rather than one large combined file?

When to Use Rules vs. CLAUDE.md

30. Why does a 500-line root CLAUDE.md mixing Terraform, Kubernetes, and CI/CD conventions "waste tokens" every time Claude works in just one of those areas?

31. When should subdirectory `CLAUDE.md` be used instead of `.claude/rules/` — what does "the directory structure already provides natural scoping" mean in practice?
 32. Given a scenario where a root `CLAUDE.md` has grown to 500+ lines and irrelevant rules are consuming tokens on specific file types, what is the fix?
-

C. Custom Slash Commands and Skills

33. What problem do Claude Code skills solve that `CLAUDE.md` cannot — repetitive workflows that shouldn't clutter always-loaded context?
34. Why are skills described as "on-demand," and how does that contrast with `CLAUDE.md` (always loaded) and rules (conditionally loaded by path)?

Skill File Structure

35. Where does a `SKILL.md` file live, and what determines the resulting slash command name?
36. What kind of content can a `SKILL.md` file contain (checklists, code examples, decision trees)?

Project-Level vs. User-Level Skills

37. What is the difference between a project-level skill (`.claude/skills/`) and a user-level skill (`~/.claude/skills/`) in terms of sharing and version control?
38. Given a scenario where a team needs a shared, version-controlled workflow invoked by a slash command, which path is correct — and what are the common distractors (user-level path, `settings.json`, root `CLAUDE.md`)?

Skills vs. `CLAUDE.md` vs. Rules

39. What determines *when* each mechanism loads — always, conditionally by path, or only on explicit invocation?
 40. In the 8-item code review checklist example, why does putting it in `CLAUDE.md` waste tokens, why does `.claude/rules/` with a `**/*` pattern provide no benefit over `CLAUDE.md`, and why is a skill the correct placement?
-

D. Hooks for Deterministic Enforcement

41. How do hooks differ fundamentally from `CLAUDE.md` instructions in terms of what Claude can do about them?
42. Why does Anthropic describe hooks as running "regardless of what the AI decides to do"?
43. Where do hooks live (`.claude/settings.json` vs. `.claude/settings.local.json`), and what is the difference between the two in terms of sharing?

PreToolUse Hooks

44. At what point does a `PreToolUse` hook run, and what three actions can it take on the planned tool call?
45. What happens when a `PreToolUse` hook returns exit code 2 — does this hold even in `bypassPermissions` mode or with `--dangerously-skip-permissions`?

46. Why does this behavior make **PreToolUse** hooks "the strongest enforcement mechanism available"?
47. What kinds of scenarios call for a **PreToolUse** hook (blocking destructive shell commands, validating inputs, protecting specific files)?

PostToolUse Hooks

48. At what point does a **PostToolUse** hook run, and what two pieces of information does it receive (tool input and tool response)?
49. What kinds of tasks suit a **PostToolUse** hook (auto-formatting, lint checks, audit logging)?
50. In the Prettier/Black example, what does the **matcher: "Edit|Write"** configuration ensure happens on every file edit?

Hook Matchers

51. What does a hook matcher determine, and what happens if no matcher is specified at all?
52. What is the difference between a matcher targeting **Edit**, one targeting **Edit|Write**, and one targeting a specific MCP tool pattern like **mcp__server__tool**?
53. Why does a matcher make hooks "targeted rather than blanket"?

Hooks vs. CLAUDE.md — Advisory vs. Deterministic

54. In the Prettier worked example, why does adding "IMPORTANT: Always run Prettier" to CLAUDE.md still leave roughly 15% of files inconsistently formatted?
55. Why does a **PostToolUse** hook with an **Edit|Write** matcher achieve 100% formatting compliance where the CLAUDE.md instruction could not?
56. Why is CLAUDE.md's emphasis described as improving probability but never eliminating violations, while a hook "runs outside of Claude's decision-making process" entirely?
57. Do hooks fire for subagent actions too — what happens to a hook's coverage when Claude spawns a subagent that uses tools?
58. What happens if a hook has no matcher specified — does it fire selectively, or on every tool call, and what's the cost of that?

E. Permissions and Access Controls

59. How do permissions differ from both CLAUDE.md (advisory) and hooks (scripts run around actions) — what makes them "the most restrictive enforcement mechanism"?
60. Why is a permission block described as something "Claude physically cannot circumvent"?

permissions.deny for Hard Restrictions

61. In the **Edit(./db/migrations/**)** deny example, what specifically can Claude no longer do — and how is this different from a CLAUDE.md instruction saying "don't edit migrations"?
62. How does **permissions.deny** differ from a **PreToolUse** hook in terms of implementation effort (declarative configuration vs. writing a script)?

Combining Permissions, CLAUDE.md, and Hooks

- 63. Given three requirements (never modify X, prefer Y over Z, always run W after every edit), which enforcement mechanism does each map to?
 - 64. What specific words in a requirement signal `permissions.deny` ("never"), CLAUDE.md ("prefer," "should"), and hooks ("automatically," "always" + automation)?
 - 65. In the worked example where all three requirements were mistakenly placed in CLAUDE.md, why does only one of them actually belong there?
-

F. MCP Server Configuration in Claude Code

- 66. How are MCP servers added to a Claude Code project (CLI command or direct config editing), and what is the default scope?

`.mcp.json` — Project-Level Configuration

- 67. Why is `.mcp.json` the correct place for a shared venue-lookup service or internal API that the whole team needs?
- 68. How does environment variable expansion (`${VENUE_API_KEY}`) prevent an API key from ever being committed to version control?

`~/.claude.json` — User-Level Configuration

- 69. What kind of MCP server belongs in `~/.claude.json` instead of `.mcp.json`?
- 70. Given a scenario asking to "add a shared venue lookup server for the team AND a personal experimental playlist server for yourself," which file does each belong in — and what's the common distractor?

MCP Prompts as Slash Commands

- 71. What naming convention do MCP prompts follow when surfaced in Claude Code?
- 72. Are MCP prompts automatically prepended to conversations, added to the tool registry, or surfaced as @-mentionable resources — or are they strictly explicit slash commands?
- 73. If asked how MCP prompts become accessible within Claude Code, what is the one correct answer, and why is everything else a distractor?

MCP Resources for Content Catalogs

- 74. What problem occurs when Claude is connected to an issue tracker, a documentation wiki, and a database explorer but has no visibility into what each contains?
 - 75. What is the functional difference between an MCP tool (callable, performs actions) and an MCP resource (read-only, informational)?
 - 76. If a question describes Claude making excessive sequential exploratory calls across multiple MCP servers due to lacking visibility into content, what is the fix?
-

G. Plan Mode vs. Direct Execution

- 77. What is the core difference in purpose between plan mode ("think before acting") and direct execution ("just do it")?

78. Is the choice between modes about task difficulty, or about whether the task benefits from explicit analysis before action?

When to Use Plan Mode

79. Why does an unfamiliar codebase area justify plan mode even for an experienced developer?
80. In the inventory-service concurrency bug example, what would Claude be forced to do without plan mode — and what does plan mode let it do instead before any fix is attempted?
81. Why does a task with multiple possible approaches (WebSockets vs. SSE vs. polling) call for plan mode rather than direct execution?
82. Why would a security audit or large migration require plan mode to map affected code paths first?

When to Use Direct Execution

83. What three properties make a task suited to direct execution (simple, well-scoped, low-risk)?
84. Comparing "add a date validation check to one function" against "critical bug in an unfamiliar module" — what specifically determines which mode each needs?
85. Why is "rename `getUserData` to `fetchUserProfile` everywhere" suited to direct execution despite touching many call sites?

When Multi-Phase Workflows Improve Outcomes

86. What distinguishes a task that benefits from multi-phase planning (ambiguous, complex, multiple valid approaches) from one that does not (mechanical, well-defined, single correct outcome)?
87. Why does the "improve error handling throughout a module" example benefit from multi-phase while "rename this function everywhere" does not, even though both touch many files?
88. When asked which of two requests benefits more from an explicit multi-phase workflow, is the answer ever the mechanical transformation task?

H. Iterative Development Workflows

89. What is the central insight behind choosing an iteration strategy — that different problem types require different feedback loops?
90. Why does vague feedback like "try again" or "do better" leave Claude "guessing," while test results and specific input/output pairs do not?

Test-Driven Iteration

91. Why is writing tests first, then asking Claude to pass them, described as the most effective pattern for iterative refinement?
92. In the graph-traversal example (cycles, disconnected nodes, weighted edges), why does a test failure message ("expected [A,B,D] but got [A,B,C,D]") give Claude a more precise signal than a natural-language description of the bug?
93. When a question describes implementing a complex feature and asks how to structure the workflow for efficient iterative refinement, what is the answer?

Incremental Problem Solving

- 94. When multiple issues interact (column widths, date formatting, page breaks), why is fixing them one at a time — testing after each — more reliable than batching all fixes into one prompt?
- 95. What specifically goes wrong when interacting problems are all changed simultaneously?
- 96. If a question describes interacting issues and asks for the most effective approach to iterate toward a working solution, what is the answer?

Providing Concrete Test Cases

- 97. Why is providing a concrete input/expected-output pair more effective than describing a problem in natural language ("handle edge cases better")?
- 98. Why does this reasoning parallel the reason few-shot prompting outperforms zero-shot prompting?
- 99. In the null-value data migration example, what specifically should be shared with Claude instead of a vague description?

Requirements Discovery Interview and TBD Patterns

- 100. What is the difference between the "interview" pattern and the "TBD marker" pattern for surfacing unknown requirements?
- 101. In the Redis-with-5-minute-TTL caching example, what considerations might the interview pattern surface that the developer hadn't thought of (invalidation strategy, consistency guarantees, failure modes)?
- 102. When someone has a rough idea but isn't sure what considerations a robust implementation requires, why is jumping straight to a minimal implementation the wrong move?

@References for One-Off Context

- 103. When should `@references` be used instead of adding patterns to CLAUDE.md?
- 104. In the payment-processing-module example (referencing `db_utils.py`, `error_handlers.py`, `audit_logger.py`), why is `@references` preferable to describing the patterns in natural language or adding them to CLAUDE.md for a one-off task?
- 105. If patterns are "well-documented in the team wiki" and don't need additional project-level documentation, what does that imply about whether CLAUDE.md or `@references` is appropriate?

I. Session Management

- 106. What distinct problem does each session mechanism solve — resumption (context across time), forking (context across parallel paths), sub-agent spawning (context degradation), and scratchpad files (loss of specificity)?

Resuming Named Sessions

- 107. What does `--resume <session-name>` preserve that starting fresh does not?
- 108. In the auth-deep-dive scenario, why is `--resume <name>` correct while `--continue` (most recent session) is a distractor when the developer has worked on other codebases since?

Forking Sessions for Parallel Exploration

- 109. Why does `fork_session` preserve all accumulated context from before the fork point?
- 110. In the microservice-extraction-vs-refactor-in-place example, why does forking prevent the two approaches from "contaminating" each other's context — and why would exploring sequentially in the same session fail to achieve this?

Resuming After Codebase Changes

- 111. When resuming a session after a teammate merged a PR overnight, what three steps balance efficiency with accuracy?
- 112. In the "3 of 12 files changed" scenario, why is starting fresh wasteful, why is resuming without mentioning the changes risky, and why is re-reading all 12 files unnecessary?

Scratchpad Files for Long Sessions

- 113. Why might Claude start referencing "typical rendering patterns" instead of the specific classes it discovered earlier in a 30+ minute session?
- 114. Why does a persistent scratchpad file solve this problem better than conversation history alone?
- 115. Why is switching to a larger model, periodically clearing context, or pre-generating file summaries each the wrong fix for this specific symptom?

Sub-Agent Spawning for Context Management

- 116. When pivoting from one exploration area (rendering) to a related but distinct one (physics integration), what two steps let the coordinator preserve prior findings while giving the new work a fresh context window?
- 117. Why does this pattern act as a "compressed knowledge transfer" between sessions rather than a full context copy?

The `/clear` Command

- 118. Why is `/clear` described as "a last resort, not a first step"?
- 119. In what specific situation is `/clear` actually the correct answer (accumulated incorrect assumptions that conversation can't correct)?
- 120. Why does the exam typically present `/clear` as a distractor in most other session-management scenarios?

J. Running Claude Code in CI/CD Pipelines

- 121. What does the `claude -p` (or `--print`) flag do, and why is it the standard invocation for CI/CD pipelines?

`--max-turns` and `--max-budget-usd` Flags

- 122. What specifically does `--max-turns N` prevent (runaway loops that never converge)?
- 123. What specifically does `--max-budget-usd X` cap, and why is this enforced by Claude Code itself rather than the surrounding job runner?
- 124. Given a scenario asking how to enforce iteration and cost caps "within Claude Code itself," why are `timeout-minutes` on a GitHub Actions step, a smaller model, and `--permission-mode`

`dontAsk` all incorrect answers?

--system-prompt vs. --append-system-prompt

- 125. What is the difference in effect between `--system-prompt` (replaces) and `--append-system-prompt` (adds to) the default system prompt?
- 126. What capability does Claude lose when `--system-prompt` is used — specifically regarding Read, Grep, Glob, and other built-in tools?
- 127. In the "only comments on the piped diff, never reads surrounding files" failure pattern, which flag is the likely cause, and which flag fixes it?

Permission Modes for CI

- 128. What does `--permission-mode dontAsk` do in a non-interactive CI environment, and why is it appropriate there specifically?
-

K. Automated Code Review Patterns

- 129. What is the core tension in automated code review design — balancing precision (avoiding false positives) against recall (catching real bugs)?

Reducing False Positives with CLAUDE.md

- 130. When a review consistently flags intentional patterns (force-unwrapped optionals in tests, large coordinator classes), why is documenting these in CLAUDE.md the fix rather than adjusting the model or prompt structure?
- 131. Why does Claude "reasonably" flag these patterns in the first place without project context?

Improving Recall with Few-Shot Examples

- 132. When reviews miss real bugs (low recall), why are few-shot examples of specific bug categories more effective than vague instructions to "be more thorough"?
- 133. Why do chain-of-thought prompts help reasoning but not pattern recognition in this context?
- 134. Why must recall improvements be paired with a precision constraint — what would happen if Claude were told to simply "flag everything"?

Explicit Reporting Criteria

- 135. When too many technically accurate but "not worth addressing" findings appear, why is adding explicit reporting criteria a prompt-level fix rather than a post-processing fix?

Structured Output with Confidence and Severity Metadata

- 136. Why does telling Claude to "only report high-confidence issues" fail to filter out wrong findings — what does it suppress instead?
- 137. What is the correct alternative — reporting all findings with confidence and severity tags and deferring filtering downstream?

Output Structure Design — The `detected_pattern` Field

138. When developers dismiss 35% of findings, why is a `detected_pattern` field more actionable than category-level analysis or confidence scores?
139. What does tracking dismiss rates *per pattern* let a team do that category-level analysis cannot?

Inline Reasoning with Findings

140. When the bottleneck is investigation time (developers clicking into each finding), why does requiring inline reasoning and confidence solve this without filtering findings?

Prior Review Context to Avoid Duplicates

141. When a re-run review duplicates comments on already-fixed code, why is including prior review findings in context more accurate than post-processing filters or restricting scope to newly modified files?

Severity Criteria with Concrete Examples

142. Why do concrete code examples for each severity tier calibrate Claude's judgment better than a severity lookup table in CLAUDE.md or relative severity within a single PR?
143. In the anchoring-examples worked example (Critical/Major/Minor/Skip), why does each tier need a code example rather than just a description?

Splitting Reviews into Focused Prompts

144. When adding examples for one concern type (security) hurts recall in another (API design), why does splitting into separate focused prompts eliminate this tradeoff?
145. In the 34%→41% business logic / 82%→68% API design tradeoff example, what does consolidating findings after separate focused passes achieve that one combined prompt cannot?

Agentic Reviews for Cross-File Analysis

146. When a review misses bugs involving cross-file interactions (renamed parameters, callers in unchanged files), why does the fix require redesigning the review as a turn-limited agentic task rather than just expanding the diff?
147. Why does this pattern specifically require `--append-system-prompt` instead of `--system-prompt`?

Handling Truncated JSON Output

148. When a `report_findings` tool call hits the `max_tokens` limit on a large PR, why does splitting the review into multiple API calls (each covering a subset of files) fix this better than simply increasing `max_tokens`?
149. Why would retrying with "only critical findings" or switching to markdown output both fail to solve the underlying problem?

L. Cost Optimization with the Message Batches API

150. What is the single deciding question for choosing batch versus real-time processing — does anything stop working while waiting for the result?

151. In the reference table (overnight reports, end-of-week release notes, pre-merge checks, real-time review comments, monthly billing reports, blocked pipeline steps), what determines batch versus synchronous for each?
152. How are batch results matched back to their requests — by position in the results file, or by `custom_id`?

The 50% Cost Savings Tradeoff

153. What is the tradeoff for the Message Batches API's 50% cost reduction — what latency guarantee does it lack?
154. Why can't batch processing support multi-turn tool loops within a single request — what must agentic workflows requiring tool calls use instead?

Prompt Caching with Pre-Warming

155. Why might a shared 8,000-token system prompt's cache entry expire before later requests in the same batch execute?
156. What does a `max_tokens: 0` pre-warming request at the start of a batch accomplish?
157. If low cache hit rates are concentrated on requests processed later in the batch window, why is cache pre-warming the fix rather than extending TTL or splitting into sequential batches?

M. Claude Code's Built-in Tools

158. Why are Claude Code's built-in tools part of the default system prompt — and how does this connect to why `--append-system-prompt` preserves them while `--system-prompt` strips them?

Read, Write, Edit Tools

159. Why is Read described as "non-destructive"?
160. When should Write be used over Edit, and what happens to a file's entire content when Write is used on an existing file?
161. Why must the `old_string` parameter in Edit be unique in the file, and what happens if it appears multiple times or not at all?

Read → Write Fallback When Edit Fails

162. In the 150-line file with highly repetitive content, why does Edit fail to insert a new function between two existing ones?
163. Why is Read → modify → Write "less elegant" than Edit but reliable regardless of repetition?
164. Why are an extremely long `old_string`, a Bash heredoc append, and `replace_all` all incorrect fixes for this specific failure?

Grep and Glob Tools

165. What determines whether Grep or Glob is the right tool — searching by content, or searching by name/path?

166. For "finding all files that import `@company/auth`," why is Grep correct? For "locating all `cache*.py` files," why is Glob correct?

Bash Tool

167. What range of operations does the Bash tool cover, and why is it described as capable of "almost anything"?

N. Codebase Exploration Strategies

168. Why does the right exploration strategy depend on what you're trying to learn (architecture, function usage, error origin, or task decomposition)?

Top-Down Exploration

169. In the 15-file, 8,000-line caching layer example, why does starting with imports and class hierarchy (to find the base class) work better than reading all 15 files at once?

170. Why does loading all files simultaneously threaten to "consume most of the context window, leaving little room for Claude to reason"?

Finding All Callers Including Renamed Wrappers

171. Why does simply grepping for the original function name (`calculateTax`) miss callers if the function is re-exported as `computeOrderTax` elsewhere?

172. What two-step strategy ensures no callers are missed even under renaming?

Searching for Error Messages Across Services

173. Why is grepping for the distinctive text of an error message faster than reading READMEs or grepping for generic error-handling imports?

Decomposing Open-Ended Tasks

174. In the "add comprehensive tests to 200 legacy files" example, why does mapping structure with Glob/Grep and prioritizing high-impact modules work better than starting alphabetically or reading all 200 files first?

O. The Self-Review Limitation

175. Why does Claude reviewing its own code in the same session retain a systematic bias toward confirming its earlier decisions?

176. Why is this described as "not a prompt engineering problem" but "an architectural limitation rooted in how conversation context works"?

Context Retention Bias

177. What does it mean that Claude has already "convinced itself" its approach is correct during generation — what happens when it's then asked to self-review?

178. Why is the fact that Claude "considered these cases but concluded its approach was correct" what makes the self-review limitation severe, rather than Claude simply not thinking about edge cases at all?
179. Why is this compared to why human developers benefit from external code review?

Independent Claude Instance for Review

180. Why does a second, independent Claude Code instance without the generator's reasoning context evaluate code "purely on its merits"?
181. Why are extended thinking, comprehensive test files, and explicit self-review instructions all listed as insufficient fixes for this specific limitation?
182. If a question asks which approach "directly addresses the root cause" of the self-review limitation, what is the answer?

Cross-Cutting Self-Questioning Prompts for Domain 3

183. For any scenario describing a rule that's followed "most of the time but not always" despite CLAUDE.md emphasis — is a hook (not stronger wording) the needed fix?
184. For any scenario using the word "never" for a restriction — is `permissions.deny` the mechanism being tested, rather than CLAUDE.md or a hook?
185. For any scenario describing Claude only working from a piped diff and not reading surrounding files in CI — is the `--system-prompt` vs. `--append-system-prompt` distinction the root cause?
186. For any scenario describing a task with a single unambiguous outcome versus one with multiple approaches or unclear requirements — is direct execution or plan mode the better fit?
187. For any scenario describing Claude losing track of earlier findings in a long session — is a scratchpad file the fix, rather than a bigger model or periodic `/clear`?
188. For any scenario describing subtle bugs Claude "considered but dismissed" during its own code generation — does this call for an independent review instance rather than asking Claude to look again?
189. For any scenario contrasting "searching by content" versus "searching by name" — is this testing Grep vs. Glob?
190. For any scenario describing latency-tolerant, high-volume work — is the Message Batches API the answer, and does a mention of multi-turn tool calls rule it out instead?